free-mac-voice code review

Code review: bparlette/free-mac-voice (branch `main`)

Reviewed: tarball of `main` from 2026-10-06 (files dated Oct 3), unpacked at `/workspace/work/fmv-review/free-mac-voice-main`
Scope: `free_voice.py` (7,190 lines), `speaker_id.py`, `samsung_tv.py`, `daemon_runner.py`, `service.sh`, `install.sh`, `prime_permissions.sh`, `gallery_server.py`, `requirements.txt`, `tests/`. `masterpiece_critic.py` (the screen-critic overlay) was only skimmed. Method: one pass for architecture, a second for bugs, plus dry-run experiments that call the real functions (`DRY_RUN=True`) to confirm behaviour.

---

1. Verdict

The project is ambitious and mostly well-built. Tier 0 is fast and well tested, destructive system actions (shutdown, restart, empty trash) are gated, and there's real care around mic failover and TTS caching. The weak spot is **always-listening mode**. The wake-word matcher accepts everyday words ("make", "mark", "mike", "max"...) as "Mac". Those hits skip the command gate, can fall through to Gemini, and get logged to disk forever. `--stream` mode stops firing a command after its first use. The README overstates several features: the test count, Phonon as the default, "confirms aloud first", and voice-created shell macros.

---

2. Prioritized findings

F1. Fix now: everyday words count as the wake word, with no check after

`free_voice.py:286` (`parse_wake_word`), `free_voice.py:6789-6794` (`handle_command`, wake + command branch)

The wake regex accepts `mac|mack|matt|max|mark|match|make|mike|mock|macs|mac's`, after optional greetings `hey|hi|ok|yo|a|an|and...`. I tested it:

Ambient speech	parse_wake_word ->	Tier 0 result
"Mike, close the window"	`close the window`	**close_window**
"Mark, open the door"	`open the door`	**open_app** (fuzzy match)
"Make sure you subscribe"	`sure you subscribe`	-> Tier 0.5 / Tier 1 / Tier 2
"Matt Damon stars in a new movie"	`Damon stars in a new movie`	-> Tier 1 / Tier 2

When the wake word and a command arrive in one sentence, `t = cmd` (line 6794) goes straight into the cascade. `is_voice_command` only runs on the wake-*window* follow-up path (line 6807), not here. Any TV dialogue that starts with "Make..." or "Mark..." counts as a command in `--always` mode. Speaker ID would help, but its threshold is loose (see F9).

Fix: - Limit the default variants to `mac|mack|macs` (plus greeting + mac). - Treat the looser phonetic variants (make/mark/mike/max/matt/match/mock) as a wake only when they stand alone ("Mike." -> open the window, then require the next utterance). Never accept them in the wake + command form. - In the wake + command branch, apply the same `route(t) or is_voice_command(t)` check that the window path uses. - In always mode, never fall through to Tier 1 or Tier 2 from a loose-variant wake. - Add negative tests. `TestWakeWordHardening` (tests/test_free_voice.py:2517) only has positive cases.

F2. Fix now: with a Gemini key set, always-listening sends background speech to Google and speaks the answer

`free_voice.py:6937-6944`

```
if GEMINI_API_KEY:
    if gemini_answer(t): ...        # runs even when quiet_miss=True
elif not quiet_miss:
    if ollama_answer(t): ...
```

`quiet_miss` (always mode) only blocks the *local* answerer. With a Gemini key set, every false wake from F1 that the action tiers miss is sent to Google with search grounding, and the answer is read out loud. That contradicts "Zero Audio Uploads / runs completely offline". It also contradicts the docstring's promise that "background chatter never makes the Mac talk to itself".

Fix: `if GEMINI_API_KEY and not quiet_miss:`. Or, in always mode, allow Tier 2 only when the utterance began with the exact wake word and passed `is_voice_command`, or started with a question word ("what/who/when…").

F3. Fix now: "quit it" quits a random app, and quitting never asks first, despite the README

`free_voice.py:1432-1438` (`resolve_app` step 3 substring match), `:4826` (`quit_app` regex, `partial_ok=True`), `:1511` (`act_quit_app`), `:6421` (`_tier1_confirm`, never called), **README.md:85**

Substring matching has no minimum length or stop-word filter. With a typical app list: `resolve_app("it")` gives **TextEdit**, `"a"` gives **Mail**, `"me"` gives **Messages**. "quit it" and "close it" match `^(quit|close)( the)? (app |application )?(.+)$` and quit TextEdit with no confirmation. The README says "`quit spotify` / `close all windows` … (confirms aloud first)", but neither confirms. `act_quit_app` and `act_close_all_windows` (line 1707) act right away. `_tier1_confirm` was written for this and is dead code. The comment at line 6080 says quit_app is "gated via _DESTRUCTIVE_ACTIONS", but `_DESTRUCTIVE_ACTIONS` (line 5661) doesn't include it.

Fix: - Require `len(key) >= 3` for substring matching. - Skip a pronoun/filler set (`it, that, this, them, everything, a, an, me, up, all`) and say "Which app?" instead. - Add `quit_app` and `close_all_windows` to a "confirm" set. Ask through `confirm_spoken` in always mode (or always). Push-to-talk can skip it, since that's deliberate. - Either use `_tier1_confirm` for Tier 0.5/1 risky actions (quit_app, wifi off, type_text in Terminal) or delete it. Fix the README line to match.

F4. Fix now: `--stream` mode ignores a command after its first use, and may load a test-only model

`free_voice.py:5197`, `:5090-5101` (`PartialSession`), `:5143-5145` (`ensure_ggml_model`)

- `stream_whisper_loop` creates **one** `PartialSession` for the whole process, and `fired` is never cleared. I verified this: feeding `"Mac, volume up"`, `"Mac, volume up"`, `"Mac, mute"` fires `['vol_up', 'mute_toggle']`. The second "volume up" is silently dropped, and so is every repeat of any command afterwards. **Fix:** clear `session.fired` when a new wake word is heard, when whisper-stream emits a new segment (blank line or `### Transcription N END` in `--keep`/VAD mode), or after about 1.5 s with no new text.
- `ensure_ggml_model` prefers a hard-coded `/opt/homebrew/Cellar/whisper.cpp/1.9.4/share/whisper.cpp/for-tests-` whenever the real model is missing. I believe the `for-tests-*` ggml files in whisper.cpp are tiny test models without usable weights (please confirm in your Cellar). The path is also pinned to version 1.9.4. **Fix:** remove that branch and download `ggml-{model}.bin`. Also guard against `WHISPER_MODEL` holding an MLX repo id with a `/`, which builds a broken URL.
- Smaller: stream mode skips speaker ID, the 60 s wake-window cap, and `VOICE_MIC`. whisper-stream picks its own capture device, so pass `-c <idx>`.

F5. Should fix: the two-step "Mac … (pause) … command" loses the start of the command

`free_voice.py:150` (`VOICE_WAKE_FEEDBACK` default `"both"`), `:169-175` (`acknowledge_wake`), `:1157-1175` (always-loop `is_speaking()` handling)

After a bare "Mac", the default feedback plays the chime *and* speaks "what you want" through Kokoro. While `is_speaking()` is true, the loop discards everything: `capturing = []`, `vad.reset()`. When playback ends it

drains the buffer again. Someone who starts talking at the chime, as the README says to, gets the first second or so cut off, and Tier 0 then sees "…notes" instead of "open notes".

Fix: default `VOICE_WAKE_FEEDBACK=chime`. If voice feedback stays, don't throw away a capture that VAD had already started. Better still, turn on Apple's voice-processing I/O so the mic cancels the Mac's own speech (see Enhancement 1).

F6. Should fix: slow work runs inside the audio loop

`free_voice.py:1195` (`on_utterance(audio)` called inside `always_listen_loop`'s read loop), `:6807 + :6161` (command gate called twice)

Everything after VAD runs synchronously on the thread that reads the mic: speaker ID (including a possible 17.6 MB model download, speaker_id.py:37), STT, the tev1 gate (3 s timeout), Tier 1 (12 s timeout), Gemini (20 s), Kokoro synthesis of long answers, and Samsung TV commands. `act_tv_power` launches a new Python process. `cmd_power` sleeps 1.5 + 2.5 + 2.5 s and makes 4-6 HTTPS calls, each with a 15 s timeout (samsung_tv.py:233-275). Anything said meanwhile is lost to the drain at lines 1199-1205. A wake-window follow-up that misses the regex calls `is_voice_command` twice (6807, then again inside `ollama_decision_route` at 6161), so the gate's round-trip is paid twice.

Fix: - Split into producer and consumer. The audio thread only runs VAD and pushes utterances onto a `queue.Queue`. One worker thread transcribes and routes. - Interrupt TTS when the user starts speaking (`_say_proc.terminate()` on VAD "start"). - Pass the gate result into `tier05_route`, or cache it per utterance. - Run TV actions on a background thread, and import `samsung_tv` in-process instead of spawning a new Python each time.

F7. Should fix: background speech and typed text are kept on disk forever

`free_voice.py:6813` (`log(f"ignored (no wake word ...): {t!r}")`), `:6824`, `:317-420` (`record_attempt` / `_update_review_markdown`), **service.sh:24-26**

- In always mode, every overheard sentence is transcribed and written word for word to `~/.free-voice/daemon.log` (launchd stdout). Nothing rotates or caps that log, so months of room conversation pile up in plain text.
- `record_attempt` appends every command to `attempts.jsonl`, including `type <text>`. The README's own benchmark example is "type password123". Filtered command-gate text goes there too, and is copied into `command_review.md`.
- `_update_review_markdown` re-reads and re-parses the whole JSONL and rewrites the markdown **on every command**, so each command gets slower as the file grows without limit.

Fix: - Log only the length of ignored transcripts, behind a `VOICE_DEBUG_TRANSCRIPTS=1` opt-in. - Redact `type_text` and dictation payloads. - Cap `attempts.jsonl` (keep the last 500 lines, or use `RotatingFileHandler`). - Build `command_review.md` only when asked (`--review`, "show review"). - Add `newsyslog` rotation or a size check in `service.sh`.

F8. Should fix: no startup permission check, and the background service may lack permissions

`free_voice.py:7136-7140` (main starts listening without checks), **prime_permissions.sh:40**, **service.sh** (LaunchAgent runs `.venv/bin/python` directly)

- If microphone access is denied, CoreAudio hands back silent buffers instead of an error. Always mode then never detects speech, and push-to-talk transcribes silence. Accessibility failures quietly fall back to `osascript`, which then fails too (as the comment at line 1911 explains).
- `prime_permissions.sh` grants access "for the terminal app you're running this in". The launchd service runs the venv Python with no terminal behind it, so macOS treats it as a different app. That app needs its own Microphone, Accessibility and Input Monitoring grants, so the service can start up deaf with no visible error.

Fix: add a `check_permissions()` at startup and in `act_status`: - `AVCaptureDevice.authorizationStatusForMediaType_(AV`
- `AXIsProcessTrustedWithOptions({kAXTrustedCheckOptionPrompt: True})` - `CGPreflightScreenCaptureAccess()`
- `IOHIDCheckAccess(kIOHIDRequestTypeListenEvent)` for push-to-talk

Also flag about 3 s of exact-zero samples as "mic blocked". On failure, speak a clear message, send a notification, and open the right Settings pane. Have `service.sh install` run the prompts through the same binary launchd

will use.

F9. Should fix: voice matching turns itself off on any error, and its threshold is very loose

speaker_id.py:18 (`SPEAKER_THRESHOLD=0.17`), **:130-136** (`verify_speaker` returns `True, 1.0` on any exception), **:14, :37-50** (model downloaded from a third-party HF repo with no checksum)

- If the ONNX session fails (missing `onnxruntime`, which comes in only indirectly via `kokoro-onnx`, a corrupt model, or a failed download), every utterance is accepted and nothing is logged. The feature the README sells for TV rejection is silently off.
- A cosine threshold of 0.17 on pyannote embeddings is very permissive. Different speakers often score above that, though this is my judgment and not something I measured.
- Enrollment is 2 4 s clips. `ensure_model` can trigger a download mid-utterance inside the audio loop, from `deepghs/pyannote-embedding-onnx`, without hash pinning.

Fix: - If a profile is enrolled and verification errors, reject the utterance and `note_error()`. - Pin a sha256 and download only during install or enroll. - Enroll 5 or more clips and store a per-user threshold, for example `min(pairwise_sim) - margin`. Expose it in `--speaker-status`.

F10. Should fix: Tier 0.5 goes quiet on Ollama older than 0.35, without saying why

`free_voice.py:129-136` (defaults `tev1:0.8b`), `:6099-6151` (`is_voice_command` blocks on any error), `:6170-6198`, `:165` (`_decision_last_failure` never used; `_decision_ok` is never set to False)

The gate and the classifier call `/v1/systemone`, which needs Ollama 0.35 or later and `tev1` pulled. install.sh:104 runs `ollama pull tev1:0.8b || true` and never checks the Ollama version. If that endpoint 404s or isn't reachable: - the gate blocks, so every wake-window follow-up that misses the regex is dropped, and so is every Tier 0.5b route; - the code then falls back to `/api/chat` with `tev1`. Tev1's model page says it "tends to reply in prose" in plain chat, so JSON parsing fails.

The user just sees "nothing happened". `keep_alive` is also missing from the systemone payloads, so the model unloads after Ollama's default idle time and the next call cold-starts.

Fix: - In install.sh, check `ollama --version` >= 0.35 and require `ollama list | grep tev1`. - At runtime, tell "endpoint missing" apart from "model said no". On 404, log once, mark the decision tier unavailable with a cooldown (use the dead `_decision_last_failure`), and in push-to-talk let the gate pass rather than block. - Add `"keep_alive": "60m"` to the systemone payloads.

F11. Should fix: unpinned dependencies, private-API imports, and fragile installs

requirements.txt, `free_voice.py:872-873`, **install.sh:73-82**

- Every dependency is `>=` and there's no lock file. For example, `xa11y>=0.4.0` is currently at 0.15.2 on PyPI, and a pre-1.0 library can change its API at any release.
- Phonon loads through **private** modules: `from fermion.transcribe import _resolve` and `from fermion._speech import backends, fetch`. Any fermion-research patch release can break it. The failure is caught, and you silently drop back to Whisper.
- `faster-whisper` (CTranslate2) installs on every Mac even though Apple Silicon uses MLX.
- The venv is built with whichever `python3` comes first on `PATH`. That could be the Command Line Tools' 3.9, and `samsung_tv.py` has no `from __future__ import annotations`, so `dict | None` crashes on import there. `pip install ... || pip install --ignore-requires-python ...` hides exactly that kind of incompatibility.

Fix: - Commit a `constraints.txt` (or `uv.lock`) with tested versions. - Create the venv with `"$(brew --prefix)/bin/python3.12" -m venv`. - Remove the `--ignore-requires-python` fallback. - Make faster-whisper an optional extra. - Wrap the fermion import with a version check, and ask upstream for a public `load_model()`.

F12. Should fix: README claims that don't match the code

- **Test count:** the badge says "223 passing" (README:10) and the testing section says "213" (README:377). The suite actually defines **332** test methods: 325 in `test_free_voice.py` and 7 in `test_audit_regressions.py`. My Linux run is under 4.
- **STT default:** the README says Phonon-2 is the default on Apple Silicon (README:50, :327). The code defaults to `mlx-whisper` (`free_voice.py:122`), and so do install.sh:155-156 and the module docstring.
- **Tier 0.5:** described as "Semantic Router ... qwen2.5:1.5b". It actually defaults to `tev1:0.8b` through `/v1/systemone`; qwen2.5 is only the planner and drawing model. The "cosine embedding" is CRC32 character n-gram feature hashing (`embed_utterance`, :5932). That's fast and fine, but it's spelling similarity, not meaning.
- **"confirms aloud first"** for quit and close-all: false (F3).
- **"when I say backup, run bash ~/backup.sh"** (README:153, :226): I tested it, and voice creation is blocked by the guard at `:3779-3784` ("Voice shortcuts cannot execute shell scripts..."). The guard is also case-sensitive: "when I say cleanup, **B**ash rm -rf /tmp/scratch" gets *saved*. It doesn't run today, because the runtime check at :3847 is case-sensitive too. But making either check case-insensitive alone would open a hole. Lower-case `parts` in both places and document that shell macros must be hand-edited into `macros.json`, or use Shortcuts instead (Enhancement 2).
- **"100% on-device" / "Samsung TV over Wi-Fi":** TV control goes through the SmartThings cloud (`samsung_tv.py:64`, `api.smartthings.com`). Gemini Tier 2 is cloud too (F2). Say so under Privacy.

---

3. Top three features to add

1. **A real wake-word detector plus echo cancellation.** Replace "transcribe everything, then regex for 'mac'" with a small always-on keyword model, for example a custom "hey Mac" in openWakeWord running on CPU at about 1 % load. Only then run STT. Capture through `AVAudioEngine` with `inputNode.setVoiceProcessingEnabled(true)` so the Mac doesn't hear its own Kokoro output. That fixes F1 and F5 at the source, lets you talk over the Mac's replies, stops transcribing the whole room (F7), and saves battery. Keep the regex as a second check.
2. **Run Apple Shortcuts by voice ("run shortcut X") as the safe replacement for shell macros.** `shortcuts list` / `shortcuts run "<name>"` exposes everything the user has built in Shortcuts.app (HomeKit, Messages, Focus modes, scripts) without `bash -c`. Fuzzy-match names with the existing `resolve_app` logic, and let macros chain `shortcut:` steps. This gives the README's "run my backup script" story a safe implementation.
3. **One table of rules per action, plus "undo that".** Today the safety rules are spread across `_DESTRUCTIVE_ACTIONS`, the unused `_tier1_confirm`, regex-only gates and README promises. Put them in a single table (`action -> auto | confirm | confirm-in-always-mode | block-from-LLM`) in `config.json`. Keep a short journal of reversible actions (previous volume, window frame, Wi-Fi state, quit app -> reopen, dark mode) so "Mac, undo that" works. That makes false triggers cheap to recover from and makes F3/F10-style gaps easy to see.

---

4. Test suite

Command: `python -m unittest discover -s tests` (Python 3.12, Linux, venv at `/workspace/work/fmv-review/venv`).

Environment	Result
Plain stdlib (no numpy)	326 ran: 11 errors (numpy imports in Tier 0.5 / VAD tests), 5 skipped
+ numpy	326 ran: 2 errors, 6 skipped
+ numpy, `NO_PROXY='*'`	**all of `test_free_voice.py` passes**; 6 skipped (4 need pynput/Quartz, 2 need Pillow)

The 2 remaining errors weren't code bugs: - `test_audit_regressions.py` (7 tests) can't import on Linux:

`masterpiece_critic.py:37 import AppKit`. It needs a Mac or an `AppKit` stub, which is a Mac-only gap. - `TestReviewP6Gallery.test_requests_do_not_trigger_cleanup` returned 404 only because the sandbox's HTTP proxy grabbed the `127.0.0.1` request. It passes with `NO_PROXY='*'`.

The module docstring says the suite "runs on Linux with stdlib only". In practice it needs numpy, and `test_audit_regressions.py` can't import without AppKit.

Quality: the tests are broad (about 215 regex patterns, routing, mocks of TV, Ollama and Gemini, and regression tests named after earlier reviews) and fast (about 1.4 s). They mostly assert *which route fires*. There are **no negative tests** for false wakes (F1), no test that `--stream` handles repeated commands (F4), and no test of the always-loop audio and TTS interaction (F5). `_IN_TESTS = "unittest" in sys.modules` (`:220`) turns off logging and the HUD in production too if any dependency imports `unittest`. A test-only environment variable is safer.

---

5. Smaller notes (not in the top 12)

- `dispatch_tier1` uses `bool(p("on", True))` (`:6526, :6540, :6560, :6562`). An LLM answer of `"on": "false"` or `"off"` becomes True, so "wifi off" can turn Wi-Fi on. Parse strings explicitly.
- `int(p("seconds") ...)` at `:6596` raises on `"ten"`. It's caught by the caller, but it aborts a chain.
- `describe_action_for_prompt` at `:3692` reads `params['minutes']`, but timers use `amount`/`unit`, so it always asks to confirm "5 minutes".
- `act_media_seek` (`:2105`) rounds to 10 s steps, so "skip 5 seconds" skips 10 s, and YouTube's arrow key is 5 s, not 10.
- `VOICE_STT_ENGINE="whisper"` is documented as faster-whisper CPU but goes down the MLX path (`:925`).
- Whisper's `initial_prompt` starts with "Mac," (`:920, :933`), which may push transcripts toward a leading "Mac" in always mode and worsen F1. This is my inference; I didn't measure it.
- `gallery_server.py:294` puts the clip filename into an `onclick` JS string without escaping. There's no Host-header check, so DNS rebinding could read screen recordings served on `127.0.0.1:8765`. The risk is low, but add a Host allow-list.
- Structure: one 7,190-line module full of globals, with two separate dispatchers (`execute_match` and `dispatch_tier1`) that keep drifting apart, as F3 shows. Splitting into `audio/`, `stt/`, `router/`, `actions/` and one action registry would make the tables in Enhancement 3 easy to build.
- The repo carries a lot of critic-overlay art, including leftovers like `assets/themes/couch_duo/actions/test_stretch_comp`. Consider moving it to a separate package so clones stay small.